UTA SUMO ROBOTICS CUP 2026

Publicar el sistema con Docker Compose

Guía paso a paso para pruebas en Windows y despliegue público en Rocky Linux

Frontend React · API FastAPI · SQL Server · Migraciones · HTTPS · WebSockets · Respaldos

Objetivo

Publicar como servicio web el frontend React, API FastAPI y SQL Server. Caddy ofrece HTTPS y Nginx reenvía API y WebSockets. SQL Server no queda expuesto directamente a Internet.

Datos iniciales

En una instalación nueva sin volúmenes previos, las migraciones crean el esquema y agregan los catálogos y el evento oficial definidos por los scripts. Los datos locales de prueba no se copian. Los volúmenes de Docker persisten la base entre reinicios.

Guía revisada: 28 de octubre de 2026

Esta guía levanta el frontend React, la API FastAPI, SQL Server, las migraciones, copias de seguridad locales y, cuando hay un dominio, HTTPS automático. Los comprobantes siguen en un volumen persistente y los reglamentos se conservan en SQL Server. API y base de datos no publican puertos en Internet; el frontend es la entrada pública.

El frontend Docker usa rutas relativas del mismo origen (`/api/...` y WebSockets). Así el navegador llama al host y protocolo con que abrió el sitio, sin guardar `localhost` ni la IP de desarrollo dentro del build. Nginx pasa esas rutas a FastAPI.

Arquitectura

```
Internet / navegador
        ■ HTTPS (Caddy opcional en perfil production)
        ▼
Frontend React compilado + Nginx :8080
        ■■■ /api/* y WebSockets ■■■ FastAPI/Uvicorn :8000
        ■                              ■■■ SQL Server (cuenta limitada)
        ■■■ SPA React

SQL Server :1433 (solo red interna)
  ■■■ volumen persistente de base
  ■■■ servicio de respaldos diarios

Servicio migrate (ejecución única al iniciar)
  ■■■ SQL Server con cuenta de migración; crea la base, el usuario limitado y aplica scripts
```

El proceso WebSocket vive en memoria de FastAPI; por ahora se debe desplegar una sola réplica de `api`. El perfil de HTTPS usa Caddy, que también reenvía WebSockets.

Requisitos del servidor

- Docker Engine y Docker Compose v2; en Windows, Docker Desktop con backend WSL2.
- Linux x86-64 (Intel/AMD) para el contenedor oficial de SQL Server; el soporte del contenedor de SQL Server no cubre ARM.
- Para SQL Server, asigna al menos 2 GB de RAM al entorno Docker; 4 GB o más es más razonable para API, frontend y DB juntos.
- En despliegue público: dominio propio, DNS `A`/`AAAA` hacia el servidor y puertos TCP 80/443 disponibles para HTTPS. No publiques 1433.
- Elige una edición de SQL Server con licencia apropiada para el uso. `Developer` sirve para desarrollo/pruebas, no para producción. `Standard` requiere licencia; evalúa `Express` solo si sus límites bastan.

Instalar Docker Compose en Rocky Linux

Usa una instalación Rocky Linux x86-64 mantenida. En el servidor ejecuta:

```
sudo dnf -y install dnf-plugins-core git firewalld
sudo dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
sudo dnf -y install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker firewalld
sudo docker version
sudo docker compose version
sudo docker run --rm hello-world
```

En Rocky, las dos monturas de solo lectura de `compose.yaml` usan la etiqueta SELinux `Z`; conserva esa opción para que Caddy y el servicio de respaldos puedan leer sus archivos con SELinux en modo enforcing. Ejecuta los comandos de Docker con `sudo` en el servidor; pertenecer al grupo `docker` concede privilegios equivalentes a root.

Abre en el firewall del servidor únicamente HTTP y HTTPS:

```
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
```

```
sudo firewall-cmd --reload
sudo firewall-cmd --list-services
```

Repite la apertura de TCP 80 y 443 en el firewall del proveedor VPS/router. No abras TCP 1433, 8000 ni 8080: SQL Server y FastAPI permanecen dentro de Compose y el puerto 8080 local solo está enlazado a `127.0.0.1`.

1. Preparar la configuración

Desde la raíz del repositorio:

```
cp .env.example .env
```

En PowerShell:

```
Copy-Item .env.example .env
```

Edita `.env` y reemplaza todos los marcadores `CAMBIAR_...` con secretos aleatorios distintos. No subas `.env` al repositorio ni compartas la contraseña de `sa`.

Genera `SECRET_KEY` y contraseñas con un gestor de secretos o Python:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

`PII_ENCRYPTION_KEY` requiere exactamente 32 bytes codificados como Base64 URL-safe:

```
python -c "import secrets,base64;
  print(base64.urlsafe_b64encode(secrets.token_bytes(32)).decode().rstrip('='))"
```

Guarda esa clave de forma segura y conserva exactamente la misma al reiniciar o actualizar el servicio. Sin ella no se podrán descifrar los datos sensibles ya guardados.

Variables que deben quedar coherentes:

Variable	Valor requerido
`DB_SERVER`	`db` dentro de Compose.
`DB_NAME`	`UTA_SUMO_ROBOTICS_CUP`, o un nombre SQL válido.
`DB_USERNAME` / `DB_PASSWORD`	Cuenta de la API; Compose crea el login y le da lectura/escritura solo en la base de la app. Usa `uta_api` como usuario.
`MIGRATION_DB_USERNAME`	`sa`; se usa solamente durante la migración y el alta del primer login de aplicación.
`MSSQL_SA_PASSWORD`	Contraseña fuerte de `sa`; SQL Server la exige al inicializarse. Compose la suministra al migrador, nunca al contenedor API.
`DB_EDITION`	Desarrollo: `Developer`. Producción: edición licenciada elegida para el servidor.
`SECRET_KEY`	Secreto aleatorio de al menos 32 caracteres para firmar JWT.
`PII_ENCRYPTION_KEY`	Clave Base64 URL-safe de 32 bytes, estable y respaldada fuera del servidor.
`GOOGLE_CLIENT_ID`	Client ID web de Google; se compila en React y también lo valida el backend. Debe ser el mismo en ambos lados.
`VITE_API_BASE_URL`	Déjalo vacío para usar el mismo host y protocolo que el sitio; es el valor recomendado para Docker en local y producción. Solo configúralo si despliegas la API en otro origen.
`PUBLIC_DOMAIN`	Host público, por ejemplo `torneo.midominio.ec`, para el perfil de producción.
`CORS_ORIGINS`	En producción, el origen exacto `https://torneo.midominio.ec`; en local, `http://localhost:8080`. No uses `*`.

En Google Cloud Console, registra como origen JavaScript el origen que verán los usuarios (por ejemplo, `https://torneo.midominio.ec`; en desarrollo, `http://localhost:8080`). No se configura URI de redirección para el botón GIS que utiliza el frontend.

El `GOOGLE_CLIENT_ID` es un identificador público del cliente OAuth, no una contraseña de correo. Usa el Client ID web de la organización y agrega el dominio publicado como origen autorizado; no copies un secreto de cliente OAuth al frontend.

SMTP es opcional. Si no se configura, los correos se guardan en `EmailOutbox` para revisión/reintento administrativo; para correo real configura `SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD` y `SMTP_FROM_EMAIL`.

2. Subir el código a Rocky Linux

Cuando el pull request con la versión que van a publicar ya esté fusionado, clona la rama principal. Si aún van a validar la rama de trabajo, sustituye `main` por `feature/backend-core`:

```
cd ~
git clone --branch main --single-branch https://github.com/Deivid05032004/ProyectoNexoraSumoBox.git
cd ProyectoNexoraSumoBox
cp .env.example .env
chmod 600 .env
nano .env
```

En `.env`, usa una contraseña fuerte para `MSSQL_SA_PASSWORD` y `DB_PASSWORD`, una clave aleatoria distinta para `SECRET_KEY` y conserva la clave Base64 de `PII_ENCRYPTION_KEY`. Configura `DB_EDITION` según la licencia elegida; cambia `PUBLIC_DOMAIN` por el dominio real, `CORS_ORIGINS=https://tu-dominio` y `GOOGLE_CLIENT_ID` por el cliente OAuth web autorizado para ese origen. Mantén `VITE_API_BASE_URL` vacío. `SMTP_*` se necesita para envío real de correos; con esos campos vacíos el backend conserva los mensajes en la bandeja de salida.

No despliegues con `DB_EDITION=Developer` en producción. `.env.example` usa Developer solo para pruebas. Elige una edición/producto y licencia adecuada; Express tiene límites de capacidad. No publiques el archivo `.env` ni pierdas `PII_ENCRYPTION_KEY`: la aplicación no podrá descifrar datos existentes si cambia.

Probar Compose en Windows antes de publicar

Instala Docker Desktop desde la página oficial (https://docs.docker.com/desktop/setup/install/windows-install/) y ábrelo con el motor Linux/WSL2 activo. Después de fusionar los cambios a `main`, en PowerShell descarga el proyecto y entra a su raíz:

```
Set-Location "$HOME\Documents"
git clone --branch main --single-branch https://github.com/Deivid05032004/ProyectoNexoraSumoBox.git
Set-Location .\ProyectoNexoraSumoBox
docker version
docker compose version
docker run --rm hello-world
Copy-Item .env.example .env
notepad .env
```

Si el equipo ya tiene el repositorio descargado, usa `Set-Location C:\ruta\ProyectoSumoBox` con la ruta real en vez de volver a clonarlo. Todos los comandos de Compose se ejecutan desde la carpeta raíz, donde está `compose.yaml`.

En el `.env` local puedes usar `DB_EDITION=Developer`; crea secretos propios, deja `VITE_API_BASE_URL=` vacío y conserva `CORS_ORIGINS=http://localhost:8080`. No copies el `.env` de desarrollo al servidor de producción. Luego, todavía desde la raíz:

```
docker compose up --build -d
docker compose ps
docker compose logs --tail=200 db migrate api web backup
Invoke-WebRequest http://localhost:8080/ -UseBasicParsing
```

Abre `http://localhost:8080` en esa misma computadora. Las migraciones corren automáticamente en el primer inicio. Para crear el administrador local, ejecuta desde la raíz:

```
docker compose run --rm migrate python scripts/provision_operator_user.py
```

Al terminar, detén el entorno local con `docker compose down`; esto conserva los volúmenes locales. Nunca uses `docker compose down -v` si quieres conservar esa base.

Para migrar explícitamente más adelante, reconstruye e inicia el stack normal (el servicio `migrate` aplica scripts pendientes antes del API) o ejecuta `docker compose run --rm migrate python scripts/migrate.py`. El migrador es idempotente: una migración aplicada y registrada no se vuelve a ejecutar.

3. Preparar el dominio y HTTPS

1. Reserva el dominio o configura un nombre DNS dinámico. Una IP pública permite llegar al servidor, pero un certificado de confianza pública funciona de forma normal con un dominio que resuelva a esa IP; no publiques el inicio de sesión por HTTP.
2. Crea un registro `A` hacia la IP pública. Añade `AAAA` solo si IPv6 está configurado y accesible.
3. Verifica que el dominio resuelva a la IP y que los puertos TCP 80/443 puedan llegar al servidor.
4. Caddy emite y renueva HTTPS automáticamente cuando el dominio público apunta al servidor y los puertos están accesibles. Con una IP o `localhost`, Caddy puede usar una CA local que otros dispositivos no confían automáticamente.

No hace falta instalar Apache ni Nginx en el host: Caddy, Nginx del frontend, FastAPI y SQL Server se ejecutan en contenedores.

Si cambia la red

- Si cambia la red de la computadora desde la que navegas, no hay que modificar el proyecto: entra usando el mismo dominio público. Si solo pruebas en esa computadora, `localhost` también sigue funcionando.
- Si mueves el servidor Rocky a otra red, la IP pública podría cambiar. Actualiza el DNS `A` (o el proveedor DDNS), verifica otra vez los firewalls del servidor y del proveedor/router, y confirma que TCP 80/443 lleguen al servidor. Una IP pública fija evita ese cambio; si el servidor está detrás de CGNAT, el reenvío de puertos normalmente no será suficiente y necesitarás IP pública o un túnel/VPS.
- Mantén el dominio como URL oficial. Si cambias a una URL/IP distinta, actualiza también `PUBLIC_DOMAIN`, `CORS_ORIGINS` y el origen JavaScript autorizado en Google Cloud Console. Los accesos OAuth de Google dependen del origen autorizado.
- El puerto 8080 está publicado solo en `127.0.0.1` para pruebas locales del servidor; desde otros equipos de la red no será accesible por diseño. El acceso público se hace por Caddy en 80/443.

4. Iniciar el servicio

Para desarrollo local, sin publicar el dominio:

```
docker compose up --build -d
docker compose ps
docker compose logs -f db migrate api web backup
```

Abre http://localhost:8080 (http://localhost:8080). En Windows PowerShell se usan los mismos comandos.

Para publicar en Rocky Linux con Caddy y HTTPS, asegúrate de configurar `PUBLIC_DOMAIN` y `CORS_ORIGINS` en `.env`, y luego ejecuta el perfil `production`:

```
sudo docker compose --profile production up --build -d
sudo docker compose --profile production ps
sudo docker compose --profile production logs -f db migrate api web https backup
```

Abre `https://tu-dominio`. El dominio debe resolver al servidor y los puertos 80/443 deben estar accesibles desde Internet para que Caddy emita el certificado. El endpoint de salud se consume dentro de la red de Compose; la API no queda publicada directamente. En la primera puesta en marcha, Compose inicia SQL Server, ejecuta migraciones y luego levanta API, frontend y proxy HTTPS.

5. Crear la primera cuenta administrativa

Por seguridad no se distribuye una contraseña de administrador fija ni se crea una cuenta privilegiada universal al construir la imagen. Después de iniciar los servicios, crea el operador desde una terminal interactiva:

```
sudo docker compose run --rm migrate python scripts/provision_operator_user.py
```

Escribe correo, nombre, apellido, rol `ADMIN` y una contraseña segura de 15 o más caracteres dos veces. La contraseña no se imprime ni se guarda en texto plano. Usa el correo y la contraseña que acabas de definir para entrar al panel.

Este paso se hace una vez por cada instalación. La cuenta se almacena en la base persistente. No hay una clave administrativa universal quemada en la imagen; no uses cuentas demo en un servidor público.

En Windows PowerShell, usa el mismo comando pero sin `sudo`.

6. Confirmar que la publicación funciona

Comprueba desde el servidor:

```
sudo docker compose --profile production ps
sudo docker compose --profile production logs --tail=200 db migrate api web https backup
curl -fsS https://tu-dominio/api/v1/health
curl -I https://tu-dominio/
```

La página se sirve por HTTPS desde Caddy; el build de React consume FastAPI por el mismo origen, y Caddy/Nginx conservan el upgrade de WebSocket. Comprueba iniciar sesión, crear una inscripción, subir un comprobante y revisar el marcador desde dos dispositivos antes de anunciar el sitio.

7. Migraciones y actualizaciones

Las migraciones se ejecutan automáticamente antes de iniciar FastAPI. Cada script se registra con checksum en `dbo.SchemaMigrations`; los ya aplicados no se vuelven a ejecutar. Si se agregan migraciones en el código, se reconstruye y se levanta Compose:

```
sudo docker compose --profile production up --build -d
```

Para ejecutar el migrador manualmente:

```
sudo docker compose run --rm migrate python scripts/migrate.py
```

El migrador conecta como `sa`, pero la API usa `DB_USERNAME` y solo recibe `db_datareader`/`db_datawriter`. No cambies la cuenta de la API por `sa`.

8. Estado y registros

```
sudo docker compose --profile production ps
sudo docker compose --profile production logs --tail=200 migrate api web
curl http://localhost:8080/
```

En Windows PowerShell:

```
Invoke-WebRequest http://localhost:8080/ -UseBasicParsing
docker compose --profile production ps
docker compose --profile production logs --tail=200 migrate api web
```

El frontend usa rutas de React en modo SPA. Las solicitudes `/api/...` se reenvían a FastAPI; las rutas WebSocket `/api/...` y `/ws/...` se conservan con upgrade y timeout extendido.

9. Datos persistentes y respaldos

- `sqlserver-data`: archivos de SQL Server.
- `app-storage`: comprobantes y otros archivos administrados por la API.
- `sqlserver-backups`: copia completa comprimida de SQL Server; el servicio `backup` ejecuta un respaldo diario por defecto y retiene 14 días. Ajusta `BACKUP_INTERVAL_SECONDS` y `BACKUP_RETENTION_DAYS` si lo necesitas.
- `caddy-data` y `caddy-config`: certificados y estado de Caddy.

Verifica los respaldos:

```
sudo docker compose --profile production logs --tail=100 backup
sudo docker volume ls
```

Los respaldos en un volumen del mismo servidor no protegen contra pérdida del servidor o del disco. Copia periódicamente el contenido de `sqlserver-backups` a otra máquina o almacenamiento de objetos y prueba la restauración antes de operar el evento.

`docker compose down` (en Rocky, `sudo docker compose down`) conserva los volúmenes. No uses `docker compose down -v` en un despliegue con datos: elimina la base y los archivos persistentes.

10. Parar, reanudar y actualizar

```
sudo docker compose --profile production stop
sudo docker compose --profile production start
```

Para eliminar contenedores manteniendo datos:

```
sudo docker compose --profile production down
```

Para actualizar a una versión ya fusionada en `main`, desde la carpeta del repositorio:

```
git pull --ff-only origin main
sudo docker compose --profile production up --build -d
sudo docker compose --profile production ps
```

No elimines volúmenes al actualizar. Las migraciones nuevas se ejecutan antes de que la API vuelva a arrancar.

Limitaciones operativas conocidas

- Mantén una réplica de FastAPI por el administrador WebSocket en memoria. Para escalar horizontalmente se requiere pub/sub compartido (por ejemplo Redis) y manejo de sesiones compartidas.
- No se incluye un proveedor SMTP; se conecta el servidor SMTP que la organización configure.
- SQL Server Linux en contenedor requiere arquitectura x86-64. Confirma arquitectura, memoria y licencia con el proveedor del hosting antes de elegir la máquina.
- Caddy automatiza TLS si el dominio resuelve al servidor y 80/443 están accesibles; sin dominio se usa HTTP local.
- Esta configuración organiza el despliegue, pero antes de producción se deben guardar secretos en el gestor de secretos del hosting y copiar los respaldos fuera de la máquina.

Referencias oficiales

- Instalación de Docker en Rocky Linux (https://docs.rockylinux.org/es/gemstones/containers/docker/)
- Instalación de Docker Engine en RHEL (https://docs.docker.com/engine/install/rhel/)
- SQL Server en contenedores Linux: inicio rápido (https://learn.microsoft.com/en-us/sql/linux/install-upgrade/quickstart-install-docker)
- Variables de entorno de SQL Server Linux (https://learn.microsoft.com/en-us/sql/linux/sql-server-linux-configure-environment-variables)
- Orden de inicio y dependencias saludables de Compose (https://docs.docker.com/compose/how-tos/startup-order/)